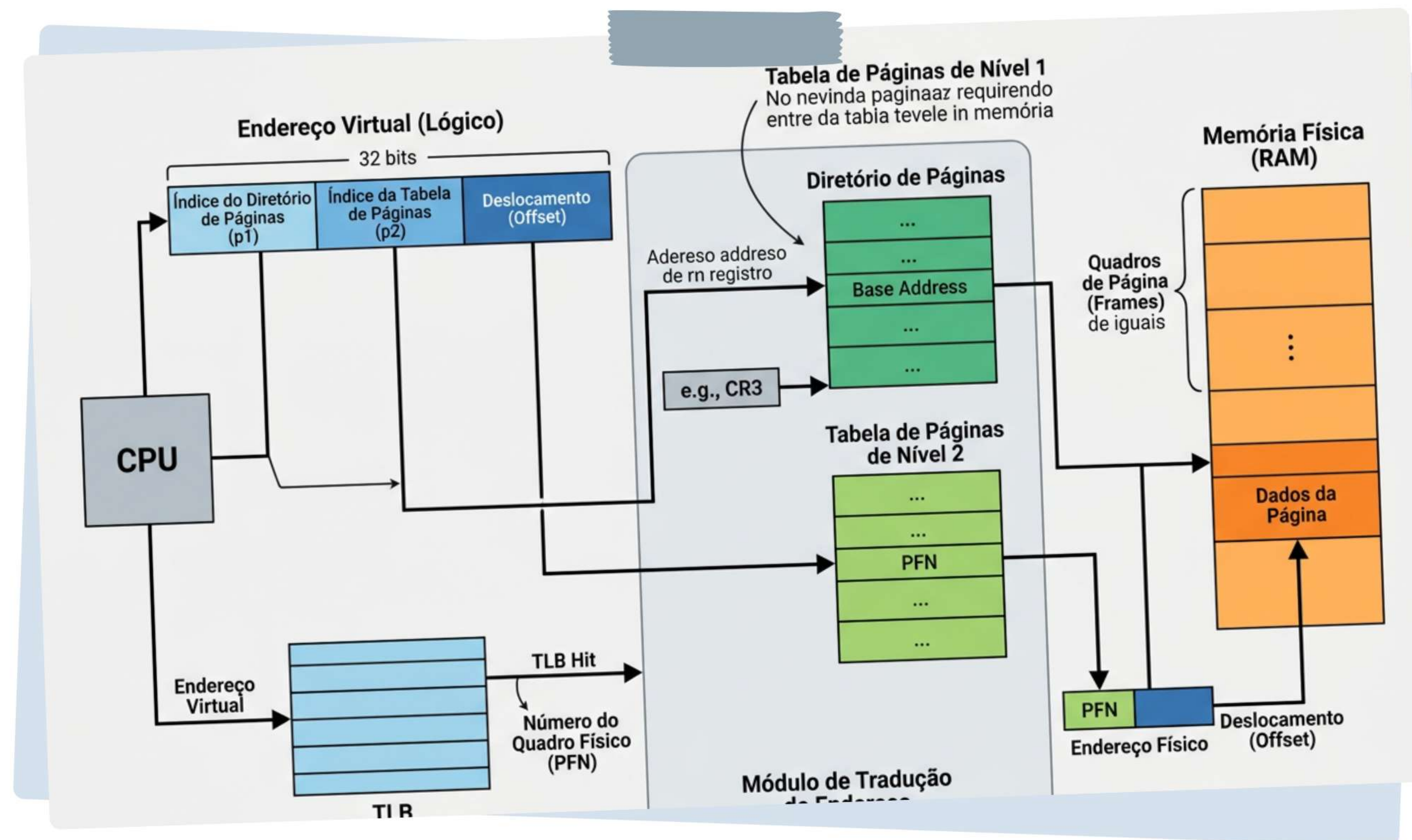


Organização e Gerenciamento da Memória Real

Fundamentos, hierarquia e estratégias de alocação no sistema operacional.



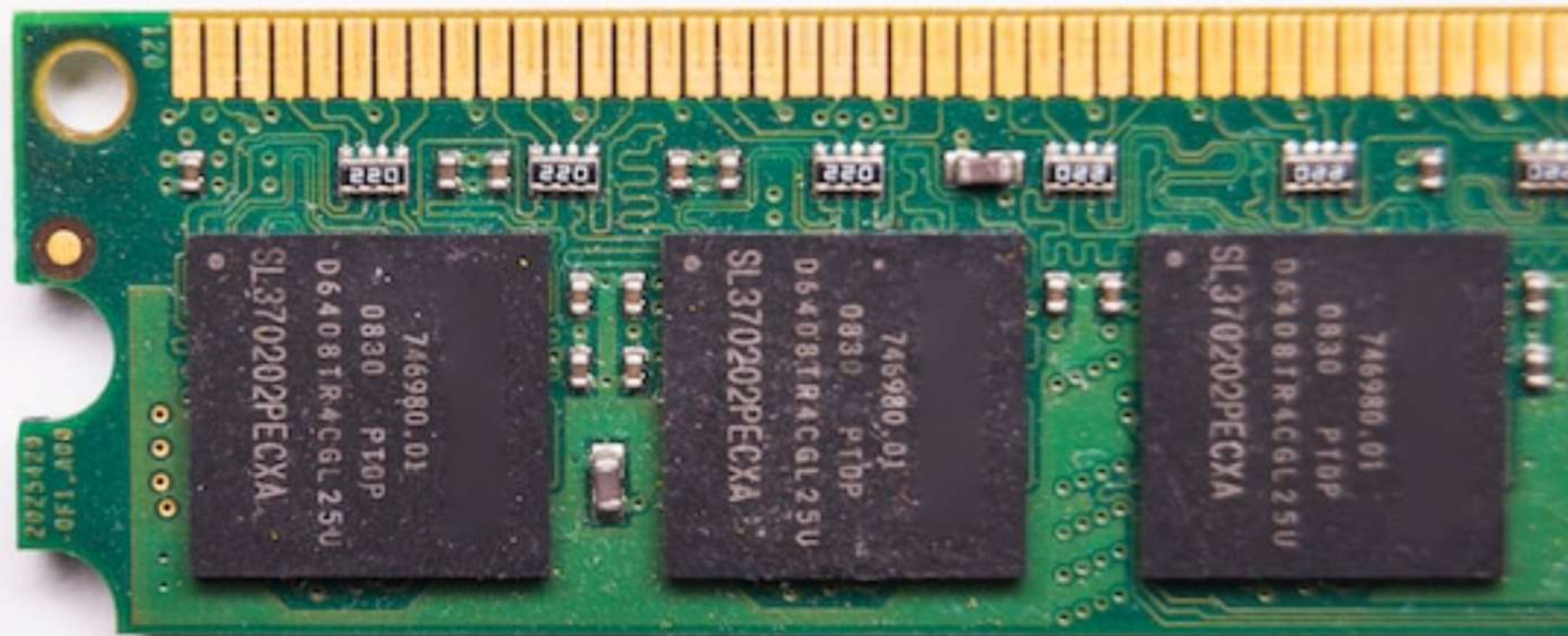
Sumário

- Introdução e desafios da memória real
- Hierarquia de memória: cache e principais
- Estratégias de gerenciamento e alocação
- Paginação, segmentação e memória virtual
- Tópicos avançados e tendências futuras



Memória Real: Introdução

Bem-vindos à nossa aula sobre a **memória real**, um dos pilares fundamentais de qualquer sistema computacional. Compreender sua organização e gerenciamento é crucial para otimizar o desempenho e a estabilidade dos sistemas modernos.

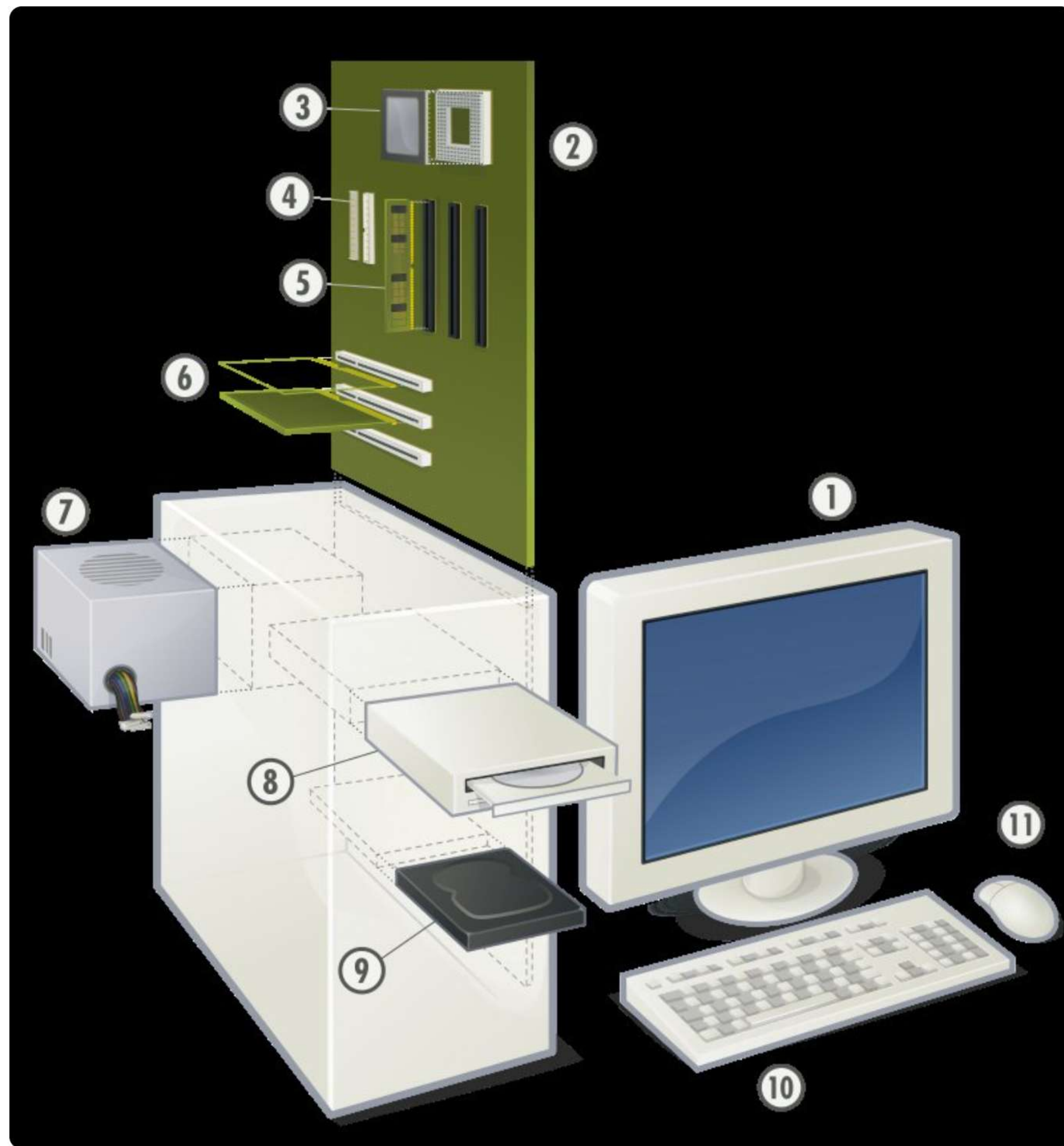


Módulo de memória RAM, essencial para a memória real do computador.



Introdução à Memória Real

- A **Memória Real** é o componente essencial para armazenamento de programas e dados, permitindo acesso direto e rápido pelo processador.
- Também designada como **Memória Principal**, ela atua como o espaço de trabalho ativo e imediato da CPU.
- Conhecida como **Memória Física**, representa a capacidade de armazenamento de hardware diretamente endereçável.
- Sendo a **Memória Primária**, sua existência é crucial, influenciando diretamente o projeto e a funcionalidade dos sistemas operacionais.



Componentes do computador, incluindo a memória principal (RAM).



Memória Real

Localizada próxima ao **processador** para acesso direto e **rápido**. É essencial para a execução de programas ativos, mas possui um custo de fabricação mais elevado por *byte*.

Armazenamento Secundário

Dispositivos como HDDs e SSDs, oferecendo capacidade massiva e **baixo custo**. São lentos e não são diretamente acessíveis ao processador, servindo primariamente para *backup* e arquivamento de dados.



Memória Principal vs. Secundária

Historicamente, a **memória principal** sempre foi mais cara que a secundária, impactando fortemente o design dos sistemas. Com o tempo, sistemas operacionais passaram a utilizá-la intensamente. Sua organização e gerenciamento tornaram-se aspectos críticos do projeto de sistema, apesar da queda de preços.



Desafios da Organização da Memória



Multiprogramação: Processos Simultâneos

A questão central é se devemos alocar um único processo à memória principal por vez ou permitir que **múltiplos processos** residam e sejam executados *simultaneamente*. Essa decisão impacta diretamente o aproveitamento da CPU e o *throughput* (vazão) do sistema.



Particionamento: Espaço da Memória

Se a multiprogramação for adotada, surge o desafio do **particionamento**: como dividir a memória principal? Podemos atribuir a cada processo a mesma quantidade de espaço ou segmentar a memória em *partições de tamanhos diferentes*, otimizando a utilização e evitando a fragmentação.

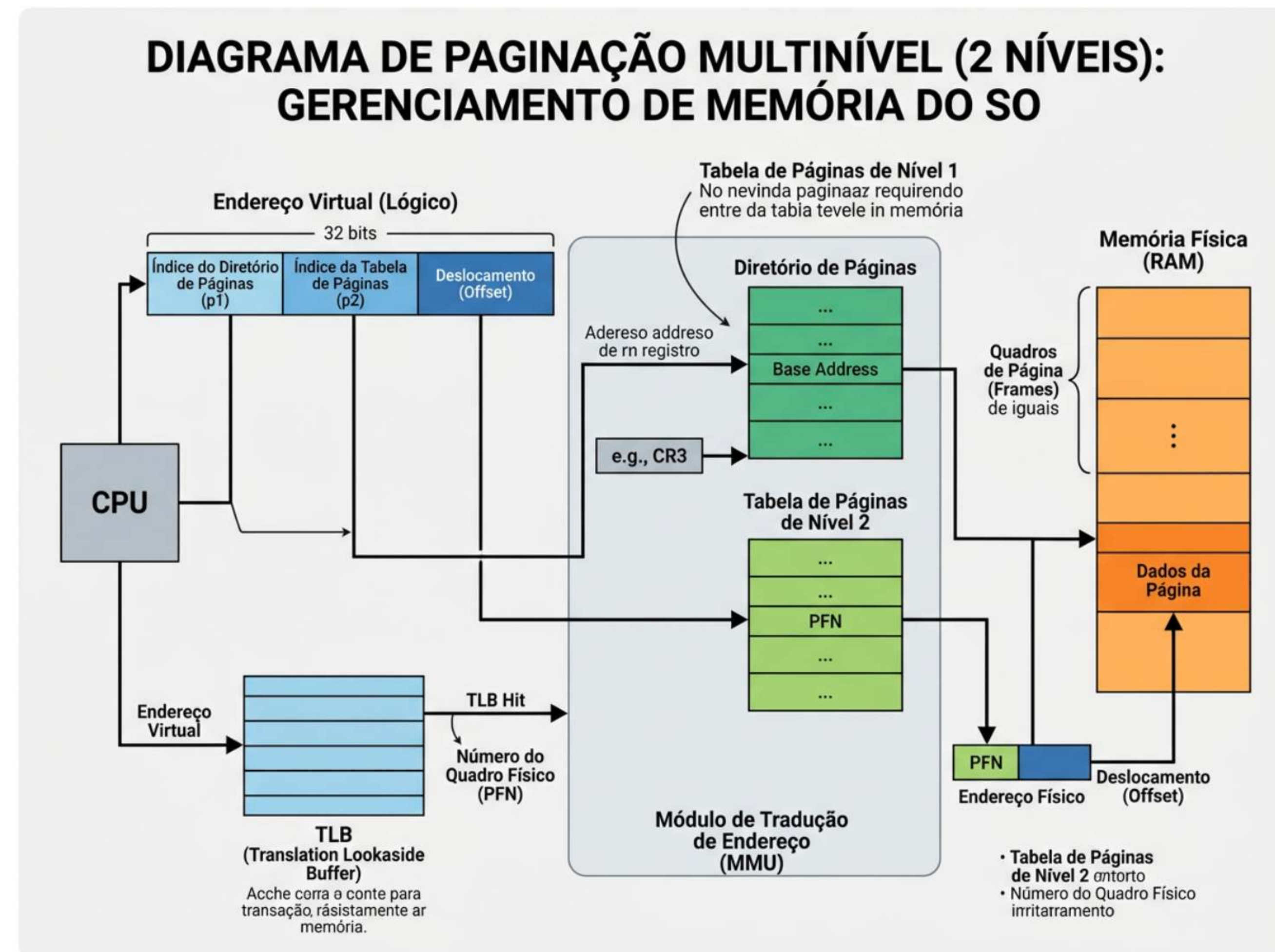


Particionamento e Alocação: Desafios

- **Definição de Partições:** As partições podem ser definidas de forma *estática* (rígida) para longos períodos ou *dinamicamente*, permitindo ao sistema adaptar-se às mudanças nas necessidades dos processos.
- **Alocação de Processos:** Sistemas podem exigir que processos executem em partições específicas ou permitir a *flexibilidade* de serem alocados onde houver espaço disponível, impactando a eficiência e fragmentação.
- **Blocos Contíguos:** A decisão de alocar processos em blocos de memória contíguos ou permitir sua divisão em blocos separados (paginação, segmentação) é crucial para gerenciar a *fragmentação externa* e a utilização da memória principal.



Gerenciamento da Memória



Paginação multinível de 2 níveis para gerenciamento de memória do SO.

- O **gerenciamento de memória** é uma função crítica do sistema operacional, normalmente controlada por software para otimização de recursos.
- Ele determina como o espaço de memória disponível é **alocado** eficientemente entre os processos em execução.
- O gerenciador de memória é responsável por **responder a mudanças** dinâmicas na utilização da memória de um processo, adaptando-se às necessidades do programa.



Requisitos de Memória: Windows

Esta tabela ilustra a evolução dos requisitos mínimos e recomendados de memória para diversas versões do Windows.

S.O.	Mínimo	Recomendado
WINDOWS 1.0	256KB	*
WINDOWS 2.03	320KB	*
WINDOWS 3.0	896KB	1MB
WINDOWS 3.1	2.6MB	4MB
WINDOWS 95	8MB	16MB
WINDOWS NT 4.0	32MB	96MB
WINDOWS 98	24MB	64MB
WINDOWS ME	32MB	128MB
WINDOWS 2000 PROFESSIONAL	64MB	128MB
WINDOWS XP HOME	64MB	128MB
WINDOWS XP PROFESSIONAL	128MB	256MB



Arquitetura de Von Neumann

Processamento Sequencial

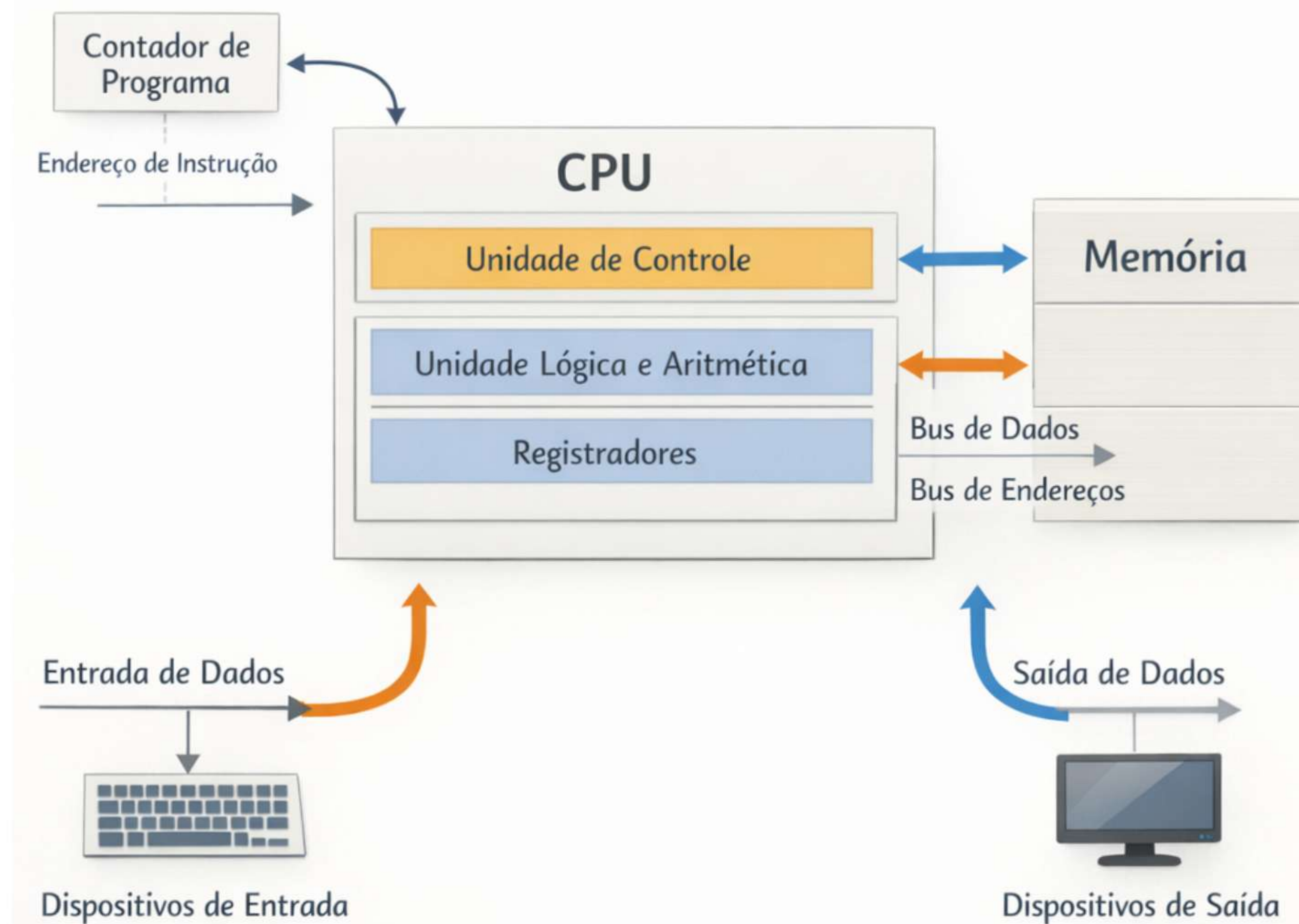


Diagrama da arquitetura Von Neumann e fluxo de dados sequencial.

Hierarquia de Memória

- A necessidade de melhorar o desempenho do sistema, frente às crescentes demandas de memória, levou à criação de uma **hierarquia de memória** com diferentes níveis de velocidade e custo.
- O nível mais alto é conhecido como **cache**, um tipo de memória muito mais rápida que a memória principal.
- A memória **cache** é estrategicamente localizada no processador, ou muito próxima a ele, para acesso rápido aos dados.
- É importante notar que a memória **cache** é extremamente cara em comparação com a memória principal, justificando seu tamanho limitado.



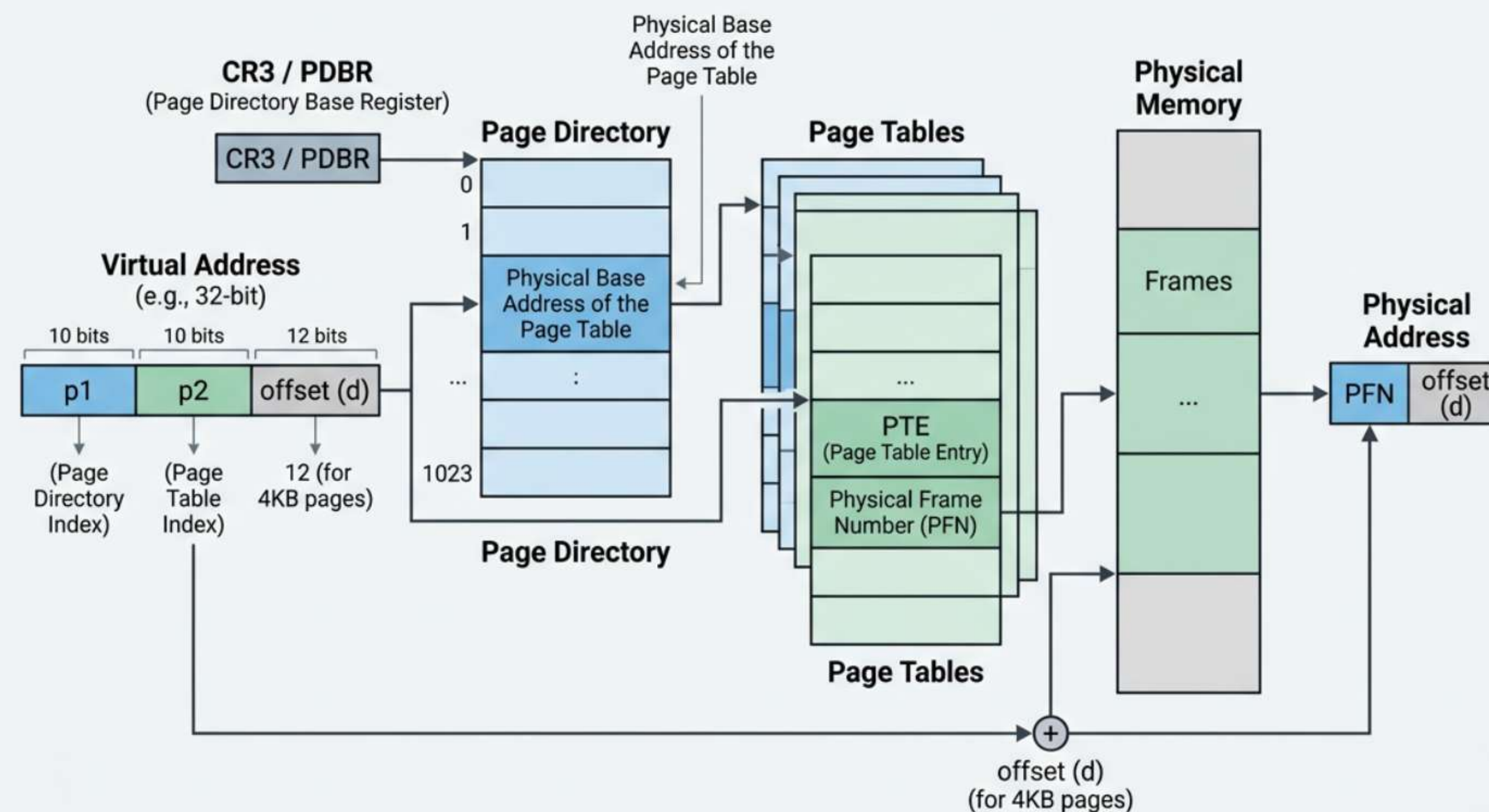
Diagrama da Hierarquia da Memória

O diagrama ilustra a hierarquia da memória, mostrando como o **processador** acessa dados e programas diretamente de seu **cache** para otimizar a velocidade. Em caso de *falha de cache*, o acesso é estendido para a memória primária e, posteriormente, para o armazenamento secundário, seguindo a ordem de latência crescente.



Estratégias de Gerenciamento

MULTILEVEL PAGING DIAGRAM (2-LEVEL PAGE TABLE SCHEME) (2-LEVEL PAGE TABLE SCHEME)



Tradução de endereço virtual para físico usando paginação de 2 níveis.

- **Estratégia de Busca:** Determina quando um bloco de dados deve ser carregado da memória secundária para a primária, otimizando o acesso antecipado ou sob demanda.
- **Estratégia de Posicionamento:** Define onde um bloco de dados será alocado na memória primária, influenciando diretamente a organização e a eficiência do mapeamento.
- **Estratégia de Substituição:** Estabelece qual bloco de dados deve ser removido da memória primária quando não há espaço disponível para um novo bloco, essencial para evitar o *thrashing* (troca excessiva de páginas).



Busca sob Demanda

Dados são transferidos para a memória principal apenas quando uma falha de página (page fault) ou segmento ocorre, indicando que são **imediatamente necessários**. Esta abordagem minimiza o consumo de memória, mas pode introduzir latência quando os dados são acessados pela primeira vez. É a base para a paginação e segmentação sob demanda.

Busca Antecipada

O sistema **prediz** quais dados serão solicitados em breve e os carrega para a memória principal proativamente, antes de serem explicitamente requisitados (pré-paginação). O objetivo é reduzir o tempo de espera futuro, mas há o risco de carregar dados desnecessários, desperdiçando recursos de memória e I/O se a previsão estiver incorreta.



Estratégias de Posicionamento

First-fit

Aloca o **primeiro bloco** de memória livre que seja grande o suficiente para o processo. É uma estratégia simples e rápida, mas pode levar a *fragmentação externa* significativa ao longo do tempo, pois os blocos menores resultantes podem não ser úteis para processos futuros, reduzindo a eficiência do uso da memória real.

Best-fit

Aloca o menor bloco de memória livre que seja grande o suficiente para o processo, visando minimizar o **desperdício** de espaço dentro do bloco alocado. Embora pareça eficiente em teoria, frequentemente leva a muitos fragmentos pequenos de memória livre, aumentando a complexidade da gerência e o tempo de busca por um bloco adequado.

Worst-fit

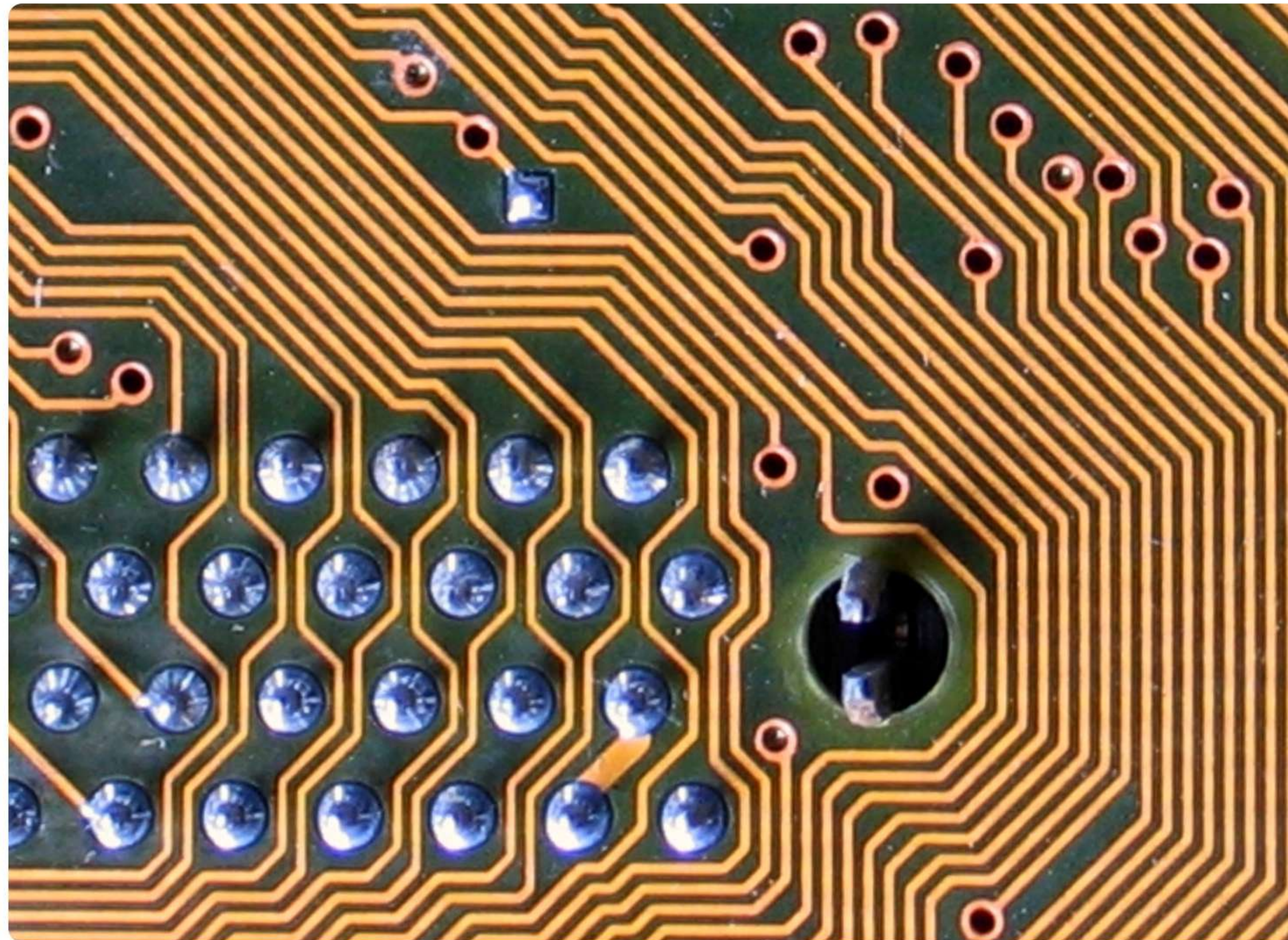
Aloca o **maior bloco** de memória livre disponível para o processo. A ideia é que, ao usar o maior bloco, os fragmentos restantes serão grandes o suficiente para serem úteis para alocações futuras. Contudo, essa estratégia pode rapidamente consumir os maiores blocos, tornando difícil encontrar espaço para processos muito grandes posteriormente e também contribuindo para a fragmentação.



Estratégia de Substituição de Páginas

- **FIFO (First-In, First-Out):** Remove a página que está na memória há mais tempo, independentemente de seu uso recente.
- **LRU (Least Recently Used):** Descarta a página que não foi referenciada pelo CPU por um período mais longo, presumindo que não será usada em breve.
- **LFU (Least Frequently Used):** Substitui a página que possui o menor contador de frequência de uso, indicando menor relevância ao longo do tempo.
- **Ótima (Optimal):** Um ideal teórico que remove a página que não será utilizada por mais tempo no futuro, impraticável na implementação real.





Placas de circuito antigas ilustram a complexidade da alocação de memória contígua.

Alocação Contígua

- **Alocação Simples:** Neste método, o sistema operacional necessitava de um bloco único e *contíguo* de memória principal para acomodar um programa por inteiro.
- **Restrição de Tamanho:** Se o espaço contíguo disponível fosse insuficiente, o programa não seria executado, o que limitava a capacidade de rodar aplicações maiores.
- **Uso Ineficiente:** Esta abordagem resultava em fragmentação interna e externa, levando a um subaproveitamento da memória e dificultando a *multiprogramação*.



Alocação Não Contígua

A Alocação Não Contígua permite que programas sejam divididos em blocos, ocupando espaços não adjacentes na memória principal. Isso possibilita um uso mais eficiente, reduzindo a fragmentação externa e permitindo a execução de programas maiores que a memória física contígua disponível.



Fragmentação Interna

A fragmentação interna manifesta-se quando a unidade de alocação de memória (bloco, página, segmento) é **maior que o espaço solicitado** pela entidade (processo, programa). O excedente de memória dentro do bloco alocado permanece não utilizado, resultando em um desperdício de recurso que, embora reservado, não pode ser empregado por outras entidades. Este fenômeno é comum em esquemas de alocação baseados em blocos de tamanho fixo ou arredondados.

Fragmentação Externa

A fragmentação externa emerge quando o montante total de memória livre é **suficiente para uma solicitação**, porém os blocos de memória disponíveis estão dispersos em pequenas porções não contíguas. Esta distribuição impede a alocação de um único bloco maior para uma nova solicitação, mesmo que a soma dos fragmentos seja adequada. É um desafio em sistemas de alocação contígua e pode ser mitigado por técnicas de compactação de memória.



Componentes da Paginação

Frames

A **memória física** é dividida em blocos de tamanho fixo, chamados *frames*. Cada frame possui um endereço físico único.

Páginas

A **memória lógica** (espaço de endereço do processo) é dividida em blocos de tamanho idêntico aos frames, conhecidos como *páginas*. O tamanho da página é crucial para a eficiência do sistema.

Tabela de Páginas

Uma **tabela de páginas** é uma estrutura de dados mantida pelo sistema operacional. Ela estabelece o mapeamento entre os endereços lógicos das páginas e os endereços físicos dos frames, permitindo acesso não contíguo.



Segmentação de Memória

- **Segmentação:** Uma técnica de alocação de memória não contígua baseada na visão *lógica* do programa, dividindo-o em unidades variáveis.
- **Segmentos:** Unidades lógicas como código, dados ou pilha, que podem ser carregadas em qualquer lugar da memória física.
- **Tabela de Segmentos:** Mantida pelo sistema operacional para cada processo, mapeando endereços lógicos para físicos, garantindo a organização.
- **Vantagem Principal:** Facilita a proteção e o compartilhamento eficiente de código e dados entre diferentes processos.



Memória Virtual

- A **Memória Virtual** cria a ilusão de que um processo tem acesso a uma memória muito maior do que a memória física realmente disponível.
- O **armazenamento secundário** (disco) é utilizado como uma extensão da memória principal, gerenciando dados entre elas.
- Baseia-se em **paginação** ou segmentação para gerenciar a troca de dados entre a memória principal e o disco (swapping/paging).
- Permite a execução de **programas maiores** do que a memória física, otimizando o uso dos recursos do sistema.
- Aumenta o **grau de multiprogramação**, permitindo que mais programas sejam executados simultaneamente.
- Simplifica o **gerenciamento de memória** para o programador, abstraindo a complexidade da memória física.





Visualização de dados em VR: gerenciamento avançado de memória e processos.

Swapping e Paging

- **Swapping:** Realiza a troca de um processo inteiro, ou de um segmento grande, entre a memória principal e o disco.
- **Paging (Paginação):** É a abordagem mais comum e granular, carregando na memória principal apenas as páginas do processo que são realmente necessárias.
- **Falha de Página:** Ocorre quando uma página requerida não está presente na memória principal, acionando o carregamento dela do disco.





Escudo de cibersegurança protege dados e rede globalmente.

Proteção de Memória

- A **estabilidade do sistema** é mantida ao impedir que um processo acesse a memória de outro sem permissão, crucial para a segurança.
- A **prevenção de corrupção** garante que nenhum processo possa danificar o sistema operacional ou outros programas em execução.
- A **implementação via hardware** é comum, utilizando Unidades de Gerenciamento de Memória (MMU) para impor essas regras.
- Os **registradores de limite e base** delimitam o intervalo de endereços que cada processo pode acessar legalmente na memória.
- As **tabelas de páginas/segmentos** incluem bits de proteção que definem permissões como leitura, escrita e execução para cada parte da memória.



Compartilhamento de Memória

- **Definição:** Permite que múltiplos processos acessem a mesma região da memória principal, otimizando o uso de recursos.
- **IPC Eficiente:** Facilita uma comunicação rápida e direta entre processos, crucial para aplicações complexas.
- **Otimização de Memória:** Reduz a duplicação de bibliotecas de código e dados comuns, economizando espaço.
- **Sincronização Necessária:** Exige mecanismos como semáforos ou monitores para coordenar acessos e evitar conflitos.
- **Segurança e Proteção:** Implementar controles para garantir que o compartilhamento seja seguro e apenas entre processos autorizados.



Alocação Dinâmica de Memória

- A **alocação dinâmica** permite que os programas solicitem e liberem memória em tempo de execução, adaptando-se às necessidades variáveis.
- O **Heap** é a região de memória dedicada a esse tipo de alocação, onde blocos de diversos tamanhos podem ser requisitados.
- Funções como ``malloc()``, ``calloc()``, ``realloc()`` e ``free()`` são essenciais para manipular a memória dinamicamente em C/C++.
- O **gerenciamento de blocos livres** é um desafio, pois o sistema precisa registrar e otimizar os espaços de memória disponíveis.
- A **fragmentação**, interna ou externa, pode ocorrer e reduzir a eficiência do uso da memória se não for adequadamente controlada.
- Os **vazamentos de memória** (memory leaks) são problemas críticos que surgem quando a memória alocada não é liberada, levando ao esgotamento de recursos.



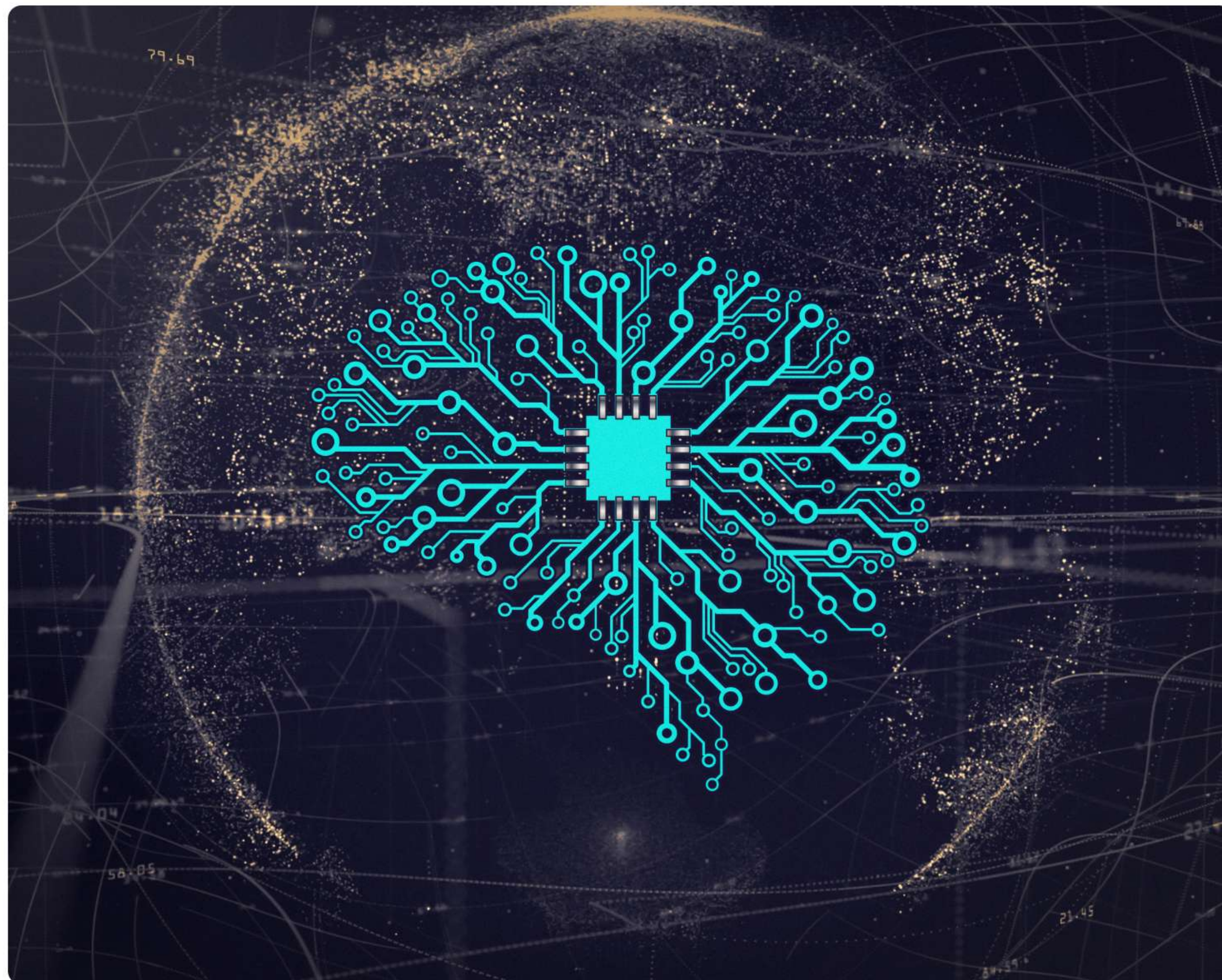


Arquitetura de sistema de gerenciamento de dados e memória

Coleta de Lixo

- **Gerenciamento Automático:** O sistema identifica e libera a memória não referenciada por objetos ou variáveis.
- **Prevenção de Erros:** Reduz vazamentos de memória e simplifica o gerenciamento para o programador.
- **Impacto no Desempenho:** Pode introduzir pausas (stalls) na execução do programa durante o processo de liberação.





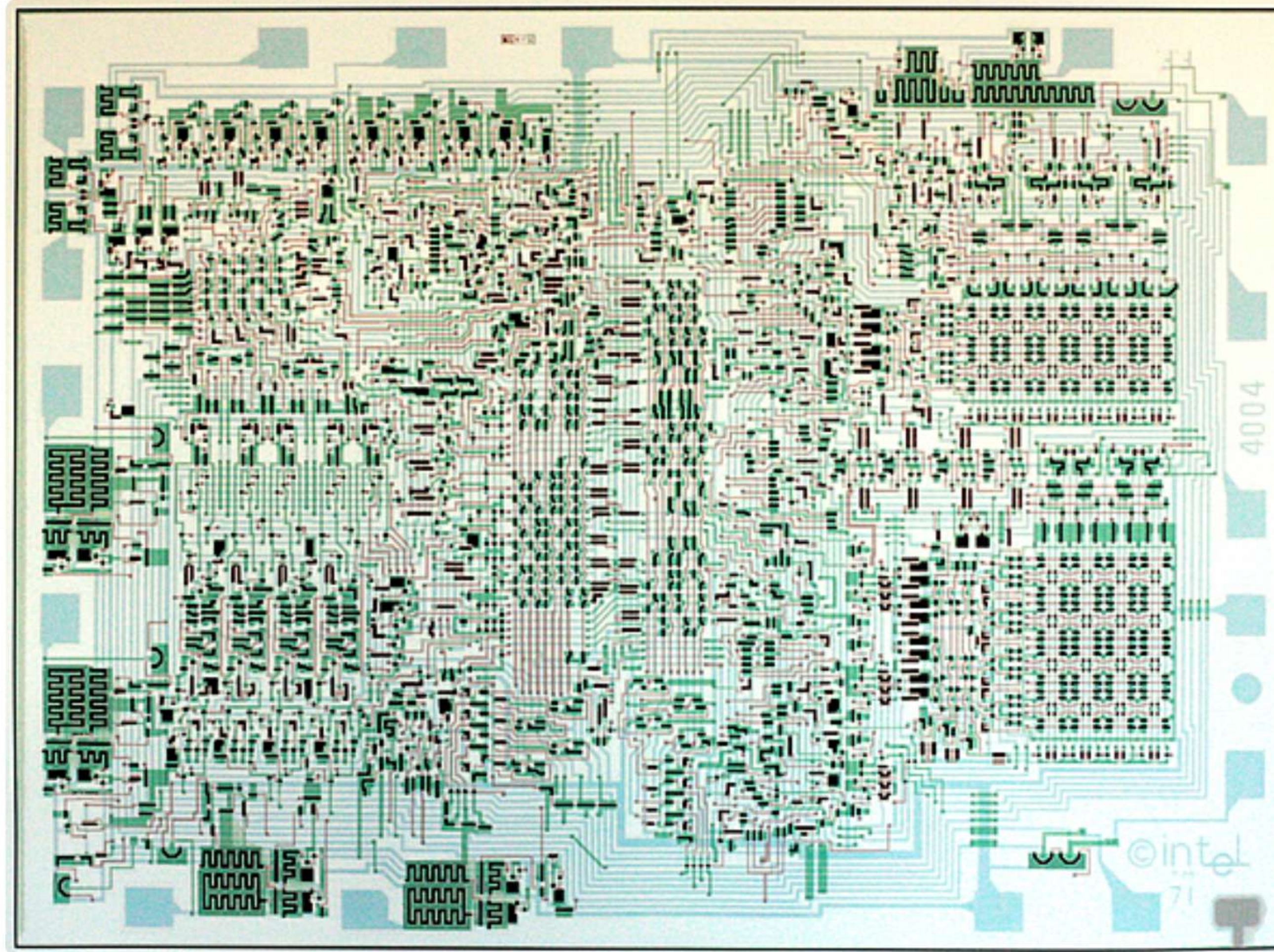
Microchip em formato de cérebro: a inteligência artificial e a computação

Memória Persistente (PMEM)

- A **PMEM** (NVM) integra a velocidade da RAM com a persistência do armazenamento secundário, redefinindo o modelo tradicional de memória.
- Com **velocidade** de DRAM, a PMEM retém os dados de forma não volátil, mesmo na ausência de energia elétrica.
- Suas **aplicações** abrangem bancos de dados de alta performance, sistemas de arquivos mais eficientes e armazenamento de dados em tempo real.
- O **impacto** dessa tecnologia promete revolucionar a arquitetura de sistemas, eliminando a distinção rígida entre memória e armazenamento.



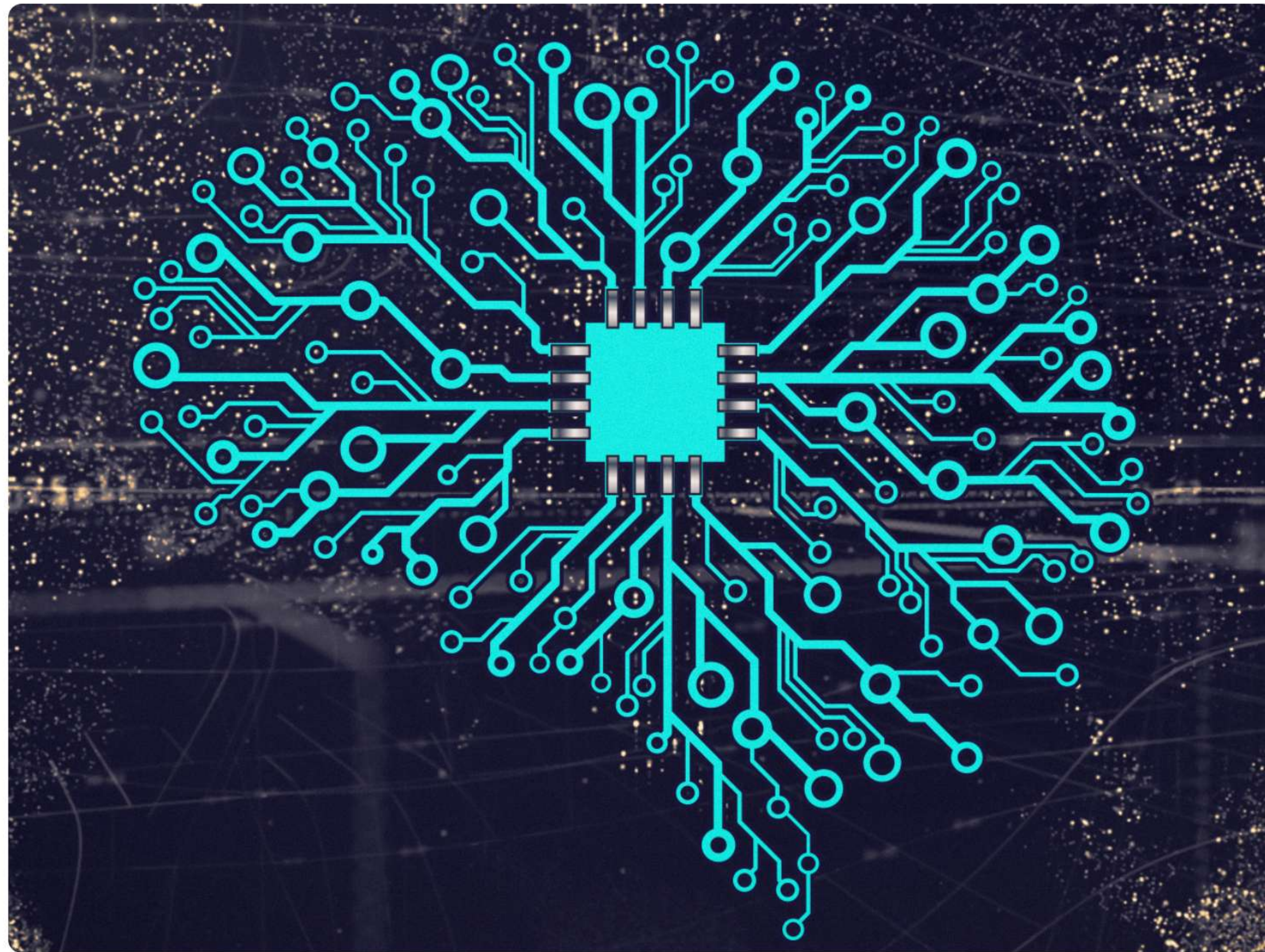
Desafios Atuais



Intel 4004: o primeiro microprocessador, precursor da computação moderna.

- A **Escalabilidade** exige gerenciamento eficaz de memória em sistemas que lidam com terabytes e milhares de núcleos de processamento.
- A **Segurança** da memória é crucial para proteger contra ataques como *Rowhammer*, *Spectre* e *Meltdown*.
- A **Heterogeneidade** desafia a integração eficiente de diversos tipos de memória, como DRAM, PMEM e GPUs.
- A **Eficiência Energética** torna-se um foco importante, visando otimizar o uso da memória para reduzir o consumo de energia.





IA no gerenciamento de memória: redes neurais no hardware.

Tendências Futuras

- **Memória Híbrida:** Combinação inteligente de diferentes tecnologias de memória, como DRAM e PMEM, otimiza custo, desempenho e persistência de dados.
- **Gerenciamento no Hardware:** Aumento da incorporação de funcionalidades de gerenciamento diretamente no hardware para garantir maior velocidade e segurança.
- **Memória Centrada em Dados:** Novas arquiteturas aproximam o processamento dos dados na própria memória, reduzindo a movimentação e latência.
- **IA no Gerenciamento:** Utilização de inteligência artificial para prever padrões de acesso e aprimorar a alocação e substituição de páginas na memória.



Revisão e Pontos Chave da Memória

- **Memória Principal:** É o sinônimo mais comum para memória real, também referida como *memória física* devido à sua natureza de hardware.
- **Memória Cache:** Situada acima da memória principal, ela armazena cópias de dados frequentemente acessados para acelerar o processamento da CPU.
- **Estratégias de Gerenciamento:** Incluem *alocação contígua*, *paginação* e *segmentação*, que organizam como os programas utilizam o espaço de memória disponível.
- **Alocação Contígua:** Diferencia-se pela reserva de um bloco único e ininterrupto de memória para um processo, enquanto a não contígua permite blocos dispersos.



Memória Real: Essência e Futuro

Compreender a organização e o gerenciamento da memória real é fundamental. Desde a hierarquia de memória até as estratégias de alocação e as tecnologias emergentes, este campo continua sendo dinâmico. A memória é crucial para o desempenho e a estabilidade dos sistemas computacionais. É essencial para qualquer profissional de tecnologia.



Conclusão

- A memória real é essencial, influenciando o design de sistemas operacionais e exigindo gerenciamento eficaz para desempenho.
- A hierarquia de memória, com cache e principal, otimiza o acesso, abordando custo vs. velocidade de dados.
- Estratégias como Busca, Posicionamento e Substituição gerenciam alocação e liberação de blocos de memória.
- Paginação, segmentação e memória virtual superam limitações da alocação contígua, permitindo programas maiores e multiprogramação.

